

Restaurant Database Management (Backend)



TEAM ID : 29

Abhishekh Krishna : 2024A7PS0269U (L1)

Arjun Kulkarni : 2024A7PS0261U (L3)

Aarav Jindal : 2024A7PS0277U (L3)

Database

```
CREATE DATABASE IF NOT EXISTS smart_food_ordering CHARACTER SET utf8mb4
COLLATE utf8mb4_general_ci;
USE smart_food_ordering;
SET NAMES utf8mb4;

-- Remove old tables first so the script can be re-run easily
DROP TABLE IF EXISTS Payment;
DROP TABLE IF EXISTS Order_Details;
DROP TABLE IF EXISTS Orders;
DROP TABLE IF EXISTS Surplus_Item;
DROP TABLE IF EXISTS Menu_Item;
DROP TABLE IF EXISTS Category;
DROP TABLE IF EXISTS Restaurant;
DROP TABLE IF EXISTS Customer;
DROP TABLE IF EXISTS Admin;

CREATE TABLE Admin (
    admin_id INT AUTO_INCREMENT PRIMARY KEY,
    full_name VARCHAR(100) NOT NULL,
    email VARCHAR(100) NOT NULL UNIQUE,
    password VARCHAR(255) NOT NULL,
    phone VARCHAR(20)
);

CREATE TABLE Customer (
    customer_id INT AUTO_INCREMENT PRIMARY KEY,
    full_name VARCHAR(100) NOT NULL,
    email VARCHAR(100) NOT NULL UNIQUE,
    password VARCHAR(255) NOT NULL,
    phone VARCHAR(20),
    address VARCHAR(255)
);

CREATE TABLE Restaurant (
    restaurant_id INT AUTO_INCREMENT PRIMARY KEY,
```

```

    restaurant_name VARCHAR(100) NOT NULL,
    owner_name VARCHAR(100),
    email VARCHAR(100) NOT NULL UNIQUE,
    phone VARCHAR(20),
    address VARCHAR(255),
    INDEX idx_restaurant_name (restaurant_name)
);

CREATE TABLE Category (
    category_id INT AUTO_INCREMENT PRIMARY KEY,
    restaurant_id INT NOT NULL,
    category_name VARCHAR(100) NOT NULL,
    FOREIGN KEY (restaurant_id) REFERENCES Restaurant(restaurant_id)
        ON DELETE CASCADE
        ON UPDATE CASCADE
);

CREATE TABLE Menu_Item (
    menu_item_id INT AUTO_INCREMENT PRIMARY KEY,
    restaurant_id INT NOT NULL,
    category_id INT NOT NULL,
    item_name VARCHAR(100) NOT NULL,
    description VARCHAR(255),
    price DECIMAL(10,2) NOT NULL,
    image_url VARCHAR(500) NOT NULL DEFAULT
'https://upload.wikimedia.org/wikipedia/commons/1/19/Butter-chicken.jpg',
    is_available TINYINT(1) NOT NULL DEFAULT 1,
    FOREIGN KEY (restaurant_id) REFERENCES Restaurant(restaurant_id)
        ON DELETE CASCADE
        ON UPDATE CASCADE,
    FOREIGN KEY (category_id) REFERENCES Category(category_id)
        ON DELETE CASCADE
        ON UPDATE CASCADE
);

CREATE TABLE Surplus_Item (
    surplus_item_id INT AUTO_INCREMENT PRIMARY KEY,

```

```

    restaurant_id INT NOT NULL,
    menu_item_id INT NOT NULL,
    quantity_available INT NOT NULL,
    surplus_price DECIMAL(10,2) NOT NULL,
    available_until DATETIME NOT NULL,
    status VARCHAR(20) NOT NULL DEFAULT 'Available',
    FOREIGN KEY (restaurant_id) REFERENCES Restaurant(restaurant_id)
        ON DELETE CASCADE
        ON UPDATE CASCADE,
    FOREIGN KEY (menu_item_id) REFERENCES Menu_Item(menu_item_id)
        ON DELETE CASCADE
        ON UPDATE CASCADE
);

```

```

CREATE TABLE Orders (
    order_id INT AUTO_INCREMENT PRIMARY KEY,
    customer_id INT NOT NULL,
    restaurant_id INT NOT NULL,
    order_date DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    order_type VARCHAR(20) NOT NULL,
    total_amount DECIMAL(10,2) NOT NULL,
    order_status VARCHAR(20) NOT NULL DEFAULT 'Pending',
    INDEX idx_orders_customer_id (customer_id),
    INDEX idx_orders_restaurant_id (restaurant_id),
    FOREIGN KEY (customer_id) REFERENCES Customer(customer_id)
        ON DELETE CASCADE
        ON UPDATE CASCADE,
    FOREIGN KEY (restaurant_id) REFERENCES Restaurant(restaurant_id)
        ON DELETE CASCADE
        ON UPDATE CASCADE
);

```

```

CREATE TABLE Order_Details (
    order_detail_id INT AUTO_INCREMENT PRIMARY KEY,
    order_id INT NOT NULL,
    menu_item_id INT NULL,
    surplus_item_id INT NULL,
    quantity INT NOT NULL,

```

```

unit_price DECIMAL(10,2) NOT NULL,
CHECK (
    (menu_item_id IS NOT NULL AND surplus_item_id IS NULL) OR
    (menu_item_id IS NULL AND surplus_item_id IS NOT NULL)
),
FOREIGN KEY (order_id) REFERENCES Orders(order_id)
    ON DELETE CASCADE
    ON UPDATE CASCADE,
FOREIGN KEY (menu_item_id) REFERENCES Menu_Item(menu_item_id)
    ON DELETE SET NULL
    ON UPDATE CASCADE,
FOREIGN KEY (surplus_item_id) REFERENCES Surplus_Item(surplus_item_id)
    ON DELETE SET NULL
    ON UPDATE CASCADE
);

CREATE TABLE Payment (
    payment_id INT AUTO_INCREMENT PRIMARY KEY,
    order_id INT NOT NULL UNIQUE,
    payment_date DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    amount_paid DECIMAL(10,2) NOT NULL,
    payment_method VARCHAR(50) NOT NULL,
    payment_status VARCHAR(20) NOT NULL DEFAULT 'Paid',
    INDEX idx_payment_order_id (order_id),
    FOREIGN KEY (order_id) REFERENCES Orders(order_id)
        ON DELETE CASCADE
        ON UPDATE CASCADE
);

```

```

INSERT INTO Admin (full_name, email, password, phone) VALUES
('System Admin', 'admin@smartfood.com', 'admin123', '9876543210');

-- Sample data for Customer
INSERT INTO Customer (full_name, email, password, phone, address) VALUES
('Amit Sharma', 'amit@gmail.com', 'amit123', '9123456780', 'Kolkata'),
('Priya Das', 'priya@gmail.com', 'priya123', '9234567890', 'Howrah');

```

```
-- Sample data for Restaurant table used as Taste of India outlets
INSERT INTO Restaurant (restaurant_name, owner_name, email, phone,
address) VALUES
('Taste of India - Park Street Outlet', 'Rakesh Gupta',
'parkstreet@tasteofindia.com', '9345678901', '18A Park Street, Kolkata,
West Bengal 700071'),
('Taste of India - Salt Lake Outlet', 'Sneha Roy',
'saltlake@tasteofindia.com', '9456789012', 'City Centre 1, DC Block,
Sector I, Bidhannagar, Kolkata, West Bengal 700064'),
('Taste of India - Gariahat Outlet', 'Arindam Sen',
'gariahat@tasteofindia.com', '9567890123', '45A Gariahat Road, Ballygunge,
Kolkata, West Bengal 700019');

-- Sample data for Category
INSERT INTO Category (restaurant_id, category_name) VALUES
(1, 'Starters'),
(1, 'Main Course'),
(1, 'Breads'),
(1, 'Rice & Biryani'),
(1, 'Desserts'),
(1, 'Beverages');

-- Sample data for Surplus_Item
INSERT INTO Surplus_Item (restaurant_id, menu_item_id, quantity_available,
surplus_price, available_until, status) VALUES
(1, 2, 10, 120.00, '2026-04-12 21:00:00', 'Available'),
(2, 32, 5, 220.00, '2026-04-12 20:30:00', 'Available'),
(3, 41, 8, 75.00, '2026-04-12 20:45:00', 'Available');

-- Sample data for Orders
INSERT INTO Orders (customer_id, restaurant_id, order_type, total_amount,
order_status) VALUES
(1, 1, 'Regular', 245.00, 'Confirmed'),
(2, 2, 'Surplus', 90.00, 'Pending');

-- Sample data for Order_Details
```

```

INSERT INTO Order_Details (order_id, menu_item_id, surplus_item_id,
quantity, unit_price) VALUES
(1, 1, NULL, 1, 220.00),
(1, 2, NULL, 1, 25.00),
(2, NULL, 2, 1, 90.00);

-- Sample data for Payment
INSERT INTO Payment (order_id, amount_paid, payment_method,
payment_status) VALUES
(1, 245.00, 'Cash on Delivery', 'Paid'),
(2, 90.00, 'UPI', 'Pending');

```

Menu Seed

```

USE smart_food_ordering;
ALTER TABLE Menu_Item
    ADD COLUMN IF NOT EXISTS image_url VARCHAR(500) NOT NULL DEFAULT
'https://upload.wikimedia.org/wikipedia/commons/1/19/Butter-chicken.jpg'
AFTER price;

ALTER TABLE Menu_Item
    MODIFY image_url VARCHAR(500) NOT NULL DEFAULT
'https://upload.wikimedia.org/wikipedia/commons/1/19/Butter-chicken.jpg';
DELETE FROM Surplus_Item;
DELETE FROM Menu_Item;
DELETE FROM Category;
ALTER TABLE Category AUTO_INCREMENT = 1;
ALTER TABLE Menu_Item AUTO_INCREMENT = 1;
INSERT INTO Category (restaurant_id, category_name) VALUES
(1, 'Starters'),
(1, 'Main Course'),
(1, 'Breads'),
(1, 'Rice & Biryani'),
(1, 'Desserts'),
(1, 'Beverages');

```

Subscription

```
CREATE TABLE Subscription (  
    subscription_id INT AUTO_INCREMENT PRIMARY KEY,  
    customer_id INT NOT NULL,  
    plan_name VARCHAR(100) NOT NULL,  
    price DECIMAL(10,2) NOT NULL,  
    start_date DATE NOT NULL,  
    end_date DATE NOT NULL,  
    status VARCHAR(20) NOT NULL DEFAULT 'Active',  
    CONSTRAINT fk_subscription_customer  
        FOREIGN KEY (customer_id) REFERENCES Customer(customer_id)  
        ON DELETE CASCADE  
        ON UPDATE CASCADE  
);  
  
INSERT INTO Subscription (customer_id, plan_name, price, start_date,  
end_date, status)  
VALUES  
    (1, 'Monthly', 199.00, '2026-04-01', '2026-04-30', 'Active');  
  
INSERT INTO Subscription (customer_id, plan_name, price, start_date,  
end_date, status)  
VALUES  
    (1, 'Premium Monthly', 499.00, '2026-05-01', '2026-05-30', 'Active');
```

Queries All Types

1.Select and Joins

```
-- advanced_sql_pack.sql  
-- Taste of India advanced SQL backend pack  
-- Database: smart_food_ordering  
-- Compatible with XAMPP / phpMyAdmin / MySQL Workbench  
  
USE smart_food_ordering;  
SET NAMES utf8mb4;
```



```

-- =====
-- 1. USEFUL SELECT QUERIES
-- =====

-- Query 1: View all customers
SELECT
    customer_id,
    full_name,
    email,
    phone,
    address
FROM Customer
ORDER BY customer_id;

-- Query 2: View all Taste of India outlets
SELECT
    restaurant_id AS outlet_id,
    restaurant_name AS outlet_name,
    owner_name AS outlet_manager,
    email,
    phone,
    address
FROM Restaurant
ORDER BY restaurant_id;

```

```

-- Query 3: View menu items with category and outlet details
SELECT
    m.menu_item_id,
    m.item_name,
    c.category_name,
    r.restaurant_name AS outlet_name,
    m.price,
    CASE
        WHEN m.is_available = 1 THEN 'Available'
        ELSE 'Not Available'
    END AS availability
FROM Menu_Item m
INNER JOIN Category c ON m.category_id = c.category_id
INNER JOIN Restaurant r ON m.restaurant_id = r.restaurant_id

```

```

ORDER BY c.category_name, m.item_name;

-- Query 4: View orders with customer and outlet names
SELECT
    o.order_id,
    c.full_name AS customer_name,
    r.restaurant_name AS outlet_name,
    o.order_date,
    o.order_type,
    o.total_amount,
    o.order_status
FROM Orders o
INNER JOIN Customer c ON o.customer_id = c.customer_id
INNER JOIN Restaurant r ON o.restaurant_id = r.restaurant_id
ORDER BY o.order_date DESC;

```

2.Nested Queries

```

-- =====
-- 2. NESTED QUERIES
-- =====

-- Nested Query 1: Customers who spent more than the average customer
-- spend
SELECT
    c.customer_id,
    c.full_name,
    SUM(o.total_amount) AS customer_total_spent
FROM Customer c
INNER JOIN Orders o ON c.customer_id = o.customer_id
GROUP BY c.customer_id, c.full_name
HAVING SUM(o.total_amount) > (
    SELECT AVG(customer_total)
    FROM (
        SELECT SUM(total_amount) AS customer_total
        FROM Orders
        GROUP BY customer_id
    ) AS customer_totals
);

```

```
-- Nested Query 2: Menu items priced above the overall average menu price
SELECT
    menu_item_id,
    item_name,
    price
FROM Menu_Item
WHERE price > (
    SELECT AVG(price)
    FROM Menu_Item
)
ORDER BY price DESC;
```

3. Correlated Queries

```
-- =====
-- 3. CORRELATED QUERIES
-- =====

-- Correlated Query 1: Customers with their individual order count
SELECT
    c.customer_id,
    c.full_name,
    (
        SELECT COUNT(*)
        FROM Orders o
        WHERE o.customer_id = c.customer_id
    ) AS total_orders
FROM Customer c
ORDER BY c.customer_id;

-- Correlated Query 2: Outlets with their latest order date
SELECT
    r.restaurant_id AS outlet_id,
    r.restaurant_name AS outlet_name,
    (
        SELECT MAX(o.order_date)
```

```

        FROM Orders o
        WHERE o.restaurant_id = r.restaurant_id
    ) AS latest_order_date
FROM Restaurant r
ORDER BY r.restaurant_id;

```

4. Function Queries

```

-- 4. FUNCTIONS
-- =====

DROP FUNCTION IF EXISTS fn_customer_total_spent;
DROP FUNCTION IF EXISTS fn_is_active_subscriber;

DELIMITER $$

CREATE FUNCTION fn_customer_total_spent(p_customer_id INT)
RETURNS DECIMAL(10,2)
DETERMINISTIC
READS SQL DATA
BEGIN
    DECLARE v_total DECIMAL(10,2);

    SELECT IFNULL(SUM(total_amount), 0)
    INTO v_total
    FROM Orders
    WHERE customer_id = p_customer_id;

    RETURN v_total;
END $$

-- Test example:
-- SELECT fn_customer_total_spent(1) AS total_spent;

CREATE FUNCTION fn_is_active_subscriber(p_customer_id INT)
RETURNS VARCHAR(3)
NOT DETERMINISTIC
READS SQL DATA
BEGIN

```

```

DECLARE v_count INT;

SELECT COUNT(*)
INTO v_count
FROM Subscription
WHERE customer_id = p_customer_id
      AND status = 'Active'
      AND end_date >= CURDATE();

IF v_count > 0 THEN
    RETURN 'Yes';
ELSE
    RETURN 'No';
END IF;
END $$

-- Test example:
-- SELECT fn_is_active_subscriber(1) AS is_subscriber;

DELIMITER ;

```

5. Procedures

```

-- 5. PROCEDURES
-- =====

DROP PROCEDURE IF EXISTS sp_get_customer_order_history;
DROP PROCEDURE IF EXISTS sp_get_outlet_menu;

DELIMITER $$

CREATE PROCEDURE sp_get_customer_order_history(IN p_customer_id INT)
BEGIN
    SELECT
        o.order_id,
        o.order_date,
        r.restaurant_name AS outlet_name,
        o.order_type,
        o.total_amount,

```

```

        o.order_status,
        p.payment_method,
        p.payment_status
FROM Orders o
INNER JOIN Restaurant r ON o.restaurant_id = r.restaurant_id
LEFT JOIN Payment p ON o.order_id = p.order_id
WHERE o.customer_id = p_customer_id
ORDER BY o.order_date DESC;
END $$

```

```

-- Test example:
-- CALL sp_get_customer_order_history(1);

```

```

CREATE PROCEDURE sp_get_outlet_menu(IN p_outlet_id INT)
BEGIN
    SELECT
        m.menu_item_id,
        m.item_name,
        c.category_name,
        m.description,
        m.price,
        CASE
            WHEN m.is_available = 1 THEN 'Available'
            ELSE 'Not Available'
        END AS availability
    FROM Menu_Item m
    INNER JOIN Category c ON m.category_id = c.category_id
    WHERE m.restaurant_id = p_outlet_id
    ORDER BY c.category_name, m.item_name;
END $$

```

```

-- Test example:
-- CALL sp_get_outlet_menu(1);

```

```

DELIMITER ;

```

6. Website Testable Triggers

```
-- 6. WEBSITE-TESTABLE TRIGGERS
-- =====

DROP TRIGGER IF EXISTS trg_orders_before_insert_defaults;
DROP TRIGGER IF EXISTS trg_payment_before_insert_defaults;
DROP TRIGGER IF EXISTS trg_subscription_before_insert_defaults;

DELIMITER $$

CREATE TRIGGER trg_orders_before_insert_defaults
BEFORE INSERT ON Orders
FOR EACH ROW
BEGIN
    IF NEW.order_date IS NULL THEN
        SET NEW.order_date = NOW();
    END IF;

    IF NEW.order_status IS NULL OR NEW.order_status = '' THEN
        SET NEW.order_status = 'Pending';
    END IF;
END $$

-- Test example:
-- checkout.php inserts into Orders during normal checkout.
-- Manual test: INSERT INTO Orders (customer_id, restaurant_id,
order_date, order_type, total_amount, order_status)
-- VALUES (1, 1, NULL, 'Regular', 250.00, NULL);

CREATE TRIGGER trg_payment_before_insert_defaults
BEFORE INSERT ON Payment
FOR EACH ROW
BEGIN
    IF NEW.payment_date IS NULL THEN
        SET NEW.payment_date = NOW();
    END IF;

    IF NEW.payment_status IS NULL OR NEW.payment_status = '' THEN
        SET NEW.payment_status = 'Paid';
    END IF;
END $
```

```

        END IF;
END $$

-- Test example:
-- checkout.php inserts into Payment after an order is placed.
-- Manual test: INSERT INTO Payment (order_id, payment_date, amount_paid,
payment_method, payment_status)
-- VALUES (1, NULL, 250.00, 'UPI', NULL);

CREATE TRIGGER trg_subscription_before_insert_defaults
BEFORE INSERT ON Subscription
FOR EACH ROW
BEGIN
    IF NEW.start_date IS NULL THEN
        SET NEW.start_date = CURDATE();
    END IF;

    IF NEW.end_date IS NULL THEN
        SET NEW.end_date = DATE_ADD(NEW.start_date, INTERVAL 30 DAY);
    END IF;

    IF NEW.status IS NULL OR NEW.status = '' THEN
        SET NEW.status = 'Active';
    END IF;
END $$

-- Test example:
-- subscription_checkout.php or the subscription flow inserts into
Subscription after payment.
-- Manual test: INSERT INTO Subscription (customer_id, plan_name, price,
start_date, end_date, status)
-- VALUES (1, 'Monthly', 199.00, CURDATE(), NULL, NULL);

DELIMITER ;

```


ERD

